

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizamos los nombres para contextualizar el problema
nombres <- c("Camilo", "Andrés", "Sofía", "Manuel", "Laura", "Carlos",
             "Daniela", "Miguel", "Valentina", "Eduardo", "Natalia",
             "José", "Isabella", "Gabriel", "Mariana")
nombre <- sample(nombres, 1)

# Aleatorizamos los tipos de recipientes
recipientes <- c("cilindro", "tubo", "conducto", "tanque cilíndrico",
                "recipiente cilíndrico", "contenedor cilíndrico")
recipiente <- sample(recipientes, 1)

# Aleatorizar adjetivos para el cilindro interno
adjetivos_interno <- c("interno", "hueco", "vacío", "interior")
adjetivo_interno <- sample(adjetivos_interno, 1)

# Aleatorizar líquidos
liquidos <- c("agua", "aceite", "combustible", "líquido", "fluido")
liquido <- sample(liquidos, 1)

# Aleatorizar lo que se desea calcular
calculos <- c("la cantidad de", "el volumen de", "cuánto", "qué cantidad de")
calculo <- sample(calculos, 1)

# Aleatorizar medidas del cilindro (manteniendo consistencia matemática)
# Diámetro externo (entre 0.3 y 0.8 metros)
d_ext <- round(runif(1, 0.3, 0.8), 1)

# Radio interno (entre 0.5 y 1.5 metros)
r_int <- round(runif(1, 0.5, 1.5), 1)

# Altura del cilindro (entre 1 y 3 metros)
altura <- round(runif(1, 1, 3), 1)

# Asegurar que las dimensiones tengan sentido (radio interno < radio externo)
while (r_int >= d_ext/2) {
  d_ext <- round(runif(1, max(r_int * 2.1, 0.3), max(r_int * 3, 0.8)), 1)
}

# Calcular radio externo
r_ext <- d_ext / 2

# Calcular el volumen necesario (es el volumen del cilindro interior)
volumen <- pi * (r_int^2) * altura

# Verificar dimensiones para coherencia física
# El diámetro externo debe ser mayor que dos veces el radio interno
if (d_ext <= 2 * r_int) {
  d_ext <- 2 * r_int + round(runif(1, 0.1, 0.5), 1)
}

```

```

  r_ext <- d_ext / 2
}

# Aleatorizar unidades de medida
unidades_longitud <- c("m", "cm", "dm")
unidad <- sample(unidades_longitud, 1)

# Ajustar valores según la unidad
factor_conversion <- 1
if (unidad == "cm") {
  factor_conversion <- 100
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
} else if (unidad == "dm") {
  factor_conversion <- 10
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
}

# Redondear todos los valores para mayor claridad
d_ext <- round(d_ext, 1)
r_int <- round(r_int, 1)
altura <- round(altura, 1)
r_ext <- round(r_ext, 1)
volumen <- round(volumen, 1)

# La respuesta correcta es "C) Altura del cilindro"
# Ya que para calcular el volumen necesitaríamos conocer la altura
opcion_correcta <- 3 # Índice de "Altura del cilindro" en el vector de opciones

# Vector de solución para r-exams (1 para la correcta, 0 para las incorrectas)
solucion <- c(0, 0, 1, 0)

```

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

```

```

# Función para generar el código TikZ del cilindro
generar_cilindro_tikz <- function(d_ext, r_int, unidad) {
  # Crear código TikZ para el cilindro
  codigo_tikz <- c(
    "\\begin{tikzpicture}[scale=0.8]",
    "% Definir estilos",
    "\\tikzset{",
    "  linea_medida/.style={gray!70, thick, <->},",
    "  flecha_linea/.style={->, thick, gray!70},",
    "  etiqueta/.style={fill=white, font=\\footnotesize}",
    "}",
    "% Cilindro externo - parte frontal visible",

```

```

"\\draw[thick] (-1.25,0) arc (180:360:1.25cm and 0.4cm);",
"\\draw[thick] (-1.25,3) arc (180:0:1.25cm and 0.4cm);",
"\\draw[thick] (-1.25,0) -- (-1.25,3);",
"\\draw[thick] (1.25,0) -- (1.25,3);",

"% Cilindro interno - parte frontal visible",
"\\draw[thick, dashed] (-0.75,0) arc (180:0:0.75cm and 0.25cm);",
"\\draw[thick] (-0.75,3) arc (180:0:0.75cm and 0.25cm);",
"\\draw[thick] (-0.75,0) -- (-0.75,3);",
"\\draw[thick] (0.75,0) -- (0.75,3);",

"% Líneas de medida",
paste0("\\draw[linea_medida] (-2,1.5) -- (2,1.5) node[midway, etiqueta] {Diámetro externo = ", d_ext, " cm};",
paste0("\\draw[linea_medida] (0,3.5) -- (0.75,3.5) node[midway, etiqueta, above] {Radio interno = ", r_int, " cm};",

"% Línea de altura",
"\\draw[linea_medida] (1.6,0) -- (1.6,3) node[midway, etiqueta, right] {Altura};",

"% Etiqueta de radio interno",
"\\draw[flecha_linea] (0.2,2.5) -- (0,2.5);",
"\\draw[flecha_linea] (0.2,2.5) -- (0.75,2.5);",

"% Etiqueta de radio externo",
"\\draw[flecha_linea] (-1.8,1) -- (-1.25,1);",

"% Líneas punteadas para ilustrar dimensiones",
"\\draw[dashed] (0,3) -- (0.75,3);",
"\\draw[dashed] (0,0) -- (0,3);",

"\\end{tikzpicture}"
)

return(codigo_tikz)
}

# Generar código TikZ para el cilindro con los valores aleatorizados
cilindro_tikz <- generar_cilindro_tikz(d_ext, r_int, unidad)

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Código Python para generar la figura 3D del cilindro basado en cilindro_v4.py
codigo_python <- paste0("
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.tri import Triangulation

# --- Parámetros del cilindro ---
radio_interno = ", r_int, "
grosor = ", r_ext - r_int, "
radio_externo = radio_interno + grosor # ", r_ext, "
altura = ", altura, " # Altura del cilindro
diametro_externo = 2 * radio_externo # ", d_ext, "
unidad = "'", unidad, "' # Unidad de medida

```

```

# --- Crear figura y ejes 3D ---
fig = plt.figure(figsize=(9, 11))
ax = fig.add_subplot(111, projection='3d')

# --- Función para crear puntos de un círculo ---
def circle_points(r, h_offset, n=100):
    theta = np.linspace(0, 2*np.pi, n)
    x = r * np.cos(theta)
    y = r * np.sin(theta)
    z = np.ones_like(theta) * h_offset
    return x, y, z

# --- Generar puntos para los círculos (bordes) ---
n_points = 100
x_top_ext, y_top_ext, z_top_ext = circle_points(radius_externo, altura, n_points)
x_top_int, y_top_int, z_top_int = circle_points(radius_interno, altura, n_points)
x_bot_ext, y_bot_ext, z_bot_ext = circle_points(radius_externo, 0, n_points)
x_bot_int, y_bot_int, z_bot_int = circle_points(radius_interno, 0, n_points)

# --- Generar malla para las superficies ---
theta_surf = np.linspace(0, 2*np.pi, 50)
z_surf = np.linspace(0, altura, 10)
theta_grid, z_grid = np.meshgrid(theta_surf, z_surf)

# Coordenadas superficie externa
x_ext_surf = radius_externo * np.cos(theta_grid)
y_ext_surf = radius_externo * np.sin(theta_grid)

# Coordenadas superficie interna
x_int_surf = radius_interno * np.cos(theta_grid)
y_int_surf = radius_interno * np.sin(theta_grid)

# --- Dibujar Superficies (EFECTO 3D) ---
surface_alpha = 0.6 # Transparencia para ver a través
outer_color = 'lightblue'
inner_color = 'lightgrey'
top_color = 'dodgerblue'

# Superficie externa
ax.plot_surface(x_ext_surf, y_ext_surf, z_grid, color=outer_color, alpha=surface_alpha,
                shade=True, rstride=1, cstride=1, zorder=1)

# Superficie interna
ax.plot_surface(x_int_surf, y_int_surf, z_grid, color=inner_color, alpha=surface_alpha,
                shade=True, rstride=1, cstride=1, zorder=1)

# Superficie anular superior (tapa)
n_radii = 10
n_angles = 50
radii = np.linspace(radius_interno, radius_externo, n_radii)
angles = np.linspace(0, 2 * np.pi, n_angles, endpoint=False)
angles = np.repeat(angles[...], n_radii, axis=1)
radii = np.repeat(radii[np.newaxis, ...], n_angles, axis=0)

```

```

x_top_annulus = (radii * np.cos(angles)).flatten()
y_top_annulus = (radii * np.sin(angles)).flatten()
z_top_annulus = np.full(x_top_annulus.shape, altura)

# Crear triangulación para la superficie anular
tri = Triangulation(x_top_annulus, y_top_annulus)
ax.plot_trisurf(x_top_annulus, y_top_annulus, z_top_annulus, triangles=tri.triangles,
               cmap=plt.get_cmap('Blues'), shade=True, alpha=0.9, zorder=1.5,
               linewidth=0)

# --- Dibujar Bordes (para definición sobre las superficies) ---
line_color = 'k'
line_width = 1.0
dashed_style = '--'
line_zorder = 2 # Asegurar que las líneas estén sobre las superficies

# Determinar qué parte del círculo está oculta (aproximado)
split_idx = n_points // 2 # Índice aproximado que separa parte frontal/trasera

# Círculo superior externo
ax.plot(x_top_ext[:split_idx+1], y_top_ext[:split_idx+1], z_top_ext[:split_idx+1],
       color=line_color, lw=line_width, zorder=line_zorder)
ax.plot(x_top_ext[split_idx:], y_top_ext[split_idx:], z_top_ext[split_idx:],
       color=line_color, lw=line_width, linestyle=dashed_style, zorder=line_zorder)

# Círculo superior interno
ax.plot(x_top_int[:split_idx+1], y_top_int[:split_idx+1], z_top_int[:split_idx+1],
       color=line_color, lw=line_width, zorder=line_zorder)
ax.plot(x_top_int[split_idx:], y_top_int[split_idx:], z_top_int[split_idx:],
       color=line_color, lw=line_width, linestyle=dashed_style, zorder=line_zorder)

# Círculo inferior externo
ax.plot(x_bot_ext[:split_idx+1], y_bot_ext[:split_idx+1], z_bot_ext[:split_idx+1],
       color=line_color, lw=line_width, zorder=line_zorder)
ax.plot(x_bot_ext[split_idx:], y_bot_ext[split_idx:], z_bot_ext[split_idx:],
       color=line_color, lw=line_width, linestyle=dashed_style, zorder=line_zorder)

# Líneas verticales externas (visibles en los bordes)
idx_front_right = np.argmax(x_top_ext[:split_idx+1]*np.cos(np.radians(-50)) +
                           y_top_ext[:split_idx+1]*np.sin(np.radians(-50)))
idx_front_left = np.argmin(x_top_ext[:split_idx+1]*np.cos(np.radians(-50)) +
                           y_top_ext[:split_idx+1]*np.sin(np.radians(-50)))
ax.plot([x_top_ext[idx_front_right], x_bot_ext[idx_front_right]],
       [y_top_ext[idx_front_right], y_bot_ext[idx_front_right]],
       [altura, 0], color=line_color, lw=line_width, zorder=line_zorder)
ax.plot([x_top_ext[idx_front_left], x_bot_ext[idx_front_left]],
       [y_top_ext[idx_front_left], y_bot_ext[idx_front_left]],
       [altura, 0], color=line_color, lw=line_width, zorder=line_zorder)

# --- ANOTACIONES ---
txt_color = 'k'
txt_size = 10
arrow_color = 'k'

```

```

arrow_lw = 0.8
anno_zorder = 5 # Asegurar que las anotaciones estén encima de todo

# --- Offsets para alejar las etiquetas ---
label_dist_factor = 1.5

# Offsets ajustados
z_offset_top_line = altura + 0.5 * label_dist_factor
z_offset_top_text = z_offset_top_line + 0.4

z_offset_bot_line = -0.3 * label_dist_factor
z_offset_bot_text = z_offset_bot_line - 0.2

x_offset_side_line = radio_externo + 0.4 * label_dist_factor
x_offset_side_text = x_offset_side_line + 0.5

# Diámetro externo
ax.text(0, 0, z_offset_top_text, f'Diámetro externo = {diametro_externo} {unidad}',
       color=txt_color, size=txt_size, ha='center', va='center', zorder=anno_zorder)
ax.plot([-radio_externo, radio_externo], [0, 0], [z_offset_top_line, z_offset_top_line],
       color=arrow_color, lw=arrow_lw, zorder=anno_zorder)
ax.plot([-radio_externo, -radio_externo], [0, 0], [z_offset_top_line, altura],
       color=arrow_color, lw=arrow_lw, zorder=anno_zorder)
ax.plot([radio_externo, radio_externo], [0, 0], [z_offset_top_line, altura],
       color=arrow_color, lw=arrow_lw, zorder=anno_zorder)

# Radio interno
text_ri_x = radio_interno * 0.7
text_ri_y = radio_interno * 1.5 * label_dist_factor
text_ri_z = altura + 0.3 * label_dist_factor
ax.text(text_ri_x, text_ri_y, text_ri_z, f'Radio interno = {radio_interno} {unidad}',
       color=txt_color, size=txt_size, ha='left', va='bottom', zorder=anno_zorder)
ax.plot([0, radio_interno], [0, 0], [altura, altura],
       color=line_color, lw=line_width, zorder=line_zorder) # Línea del radio
leader_ri_start_x = text_ri_x - 0.1
leader_ri_start_y = text_ri_y - 0.1
leader_ri_start_z = text_ri_z - 0.05
ax.plot([leader_ri_start_x, radio_interno], [leader_ri_start_y, 0], [leader_ri_start_z, altura],
       color=arrow_color, lw=arrow_lw, zorder=anno_zorder) # Leader

# Grosor
text_gr_x = -radio_externo * 0.9 * label_dist_factor
text_gr_y = -radio_externo * 0.9 * label_dist_factor
text_gr_z = altura + 0.4 * label_dist_factor
ax.text(text_gr_x, text_gr_y, text_gr_z, f'Grosor = {grosor} {unidad}',
       color=txt_color, size=txt_size, ha='center', va='center', zorder=anno_zorder)
target_gr_x = - (radio_interno + grosor / 2)
target_gr_y = 0
target_gr_z = altura
leader_gr_start_x = text_gr_x + 0.1
leader_gr_start_y = text_gr_y + 0.1
leader_gr_start_z = text_gr_z - 0.05
ax.plot([leader_gr_start_x, target_gr_x], [leader_gr_start_y, target_gr_y],

```

```

        [leader_gr_start_z, target_gr_z], color=arrow_color, lw=arrow_lw, zorder=anno_zorder)

# Altura (etiqueta sin valor numérico)
ax.text(x_offset_side_text, 0, altura / 2, 'Altura',
        color=txt_color, size=txt_size, ha='center', va='center', rotation=90, zorder=anno_zorder)
ax.plot([x_offset_side_line, x_offset_side_line], [0, 0], [0, altura],
        color=arrow_color, lw=arrow_lw, zorder=anno_zorder)
ax.plot([x_offset_side_line, radio_externo], [0, 0], [0, 0],
        color=arrow_color, lw=arrow_lw, zorder=anno_zorder)
ax.plot([x_offset_side_line, radio_externo], [0, 0], [altura, altura],
        color=arrow_color, lw=arrow_lw, zorder=anno_zorder)

# Radio externo
text_re_x = radio_externo * 0.5
text_re_y = -radio_externo * 1.2 * label_dist_factor
text_re_z = z_offset_bot_text
ax.text(text_re_x, text_re_y, text_re_z, f'Radio externo = {radio_externo} {unidad}',
        color=txt_color, size=txt_size, ha='center', va='center', zorder=anno_zorder)
ax.plot([0, radio_externo], [0, 0], [0, 0],
        color=line_color, lw=line_width, zorder=line_zorder) # Línea del radio
leader_re_start_x = text_re_x
leader_re_start_y = text_re_y + 0.1
leader_re_start_z = text_re_z + 0.05
ax.plot([leader_re_start_x, radio_externo], [leader_re_start_y, 0], [leader_re_start_z, 0],
        color=arrow_color, lw=arrow_lw, zorder=anno_zorder) # Leader

# --- Configuración visual del gráfico ---
ax.set_box_aspect([1, 1, 1]) # Proporciones iguales
ax.view_init(elev=25, azim=-50) # Ángulo de vista

# Ajustar límites para dar espacio a las etiquetas
max_extent = max(diametro_externo, altura) * label_dist_factor * 0.8
ax.set_xlim(-max_extent, max_extent)
ax.set_ylim(-max_extent, max_extent)
ax.set_zlim(-max_extent*0.3, max_extent*1.2)

ax.set_axis_off() # Ocultar ejes

# Guardar figura
plt.savefig('cilindro_3d.png', dpi=150, bbox_inches='tight')
plt.close()
")

# Ejecutar código Python para generar la figura
py_run_string(codigo_python)

```

Question

Carlos desea saber cuánta aceite se necesita para llenar el tanque cilíndrico interno, pero solamente cuenta con las medidas de las dimensiones que muestra la figura.

Alternativamente, aquí está una representación 2D del cilindro:

¿Cuál medida le falta a Carlos para hallar la cantidad deseada?

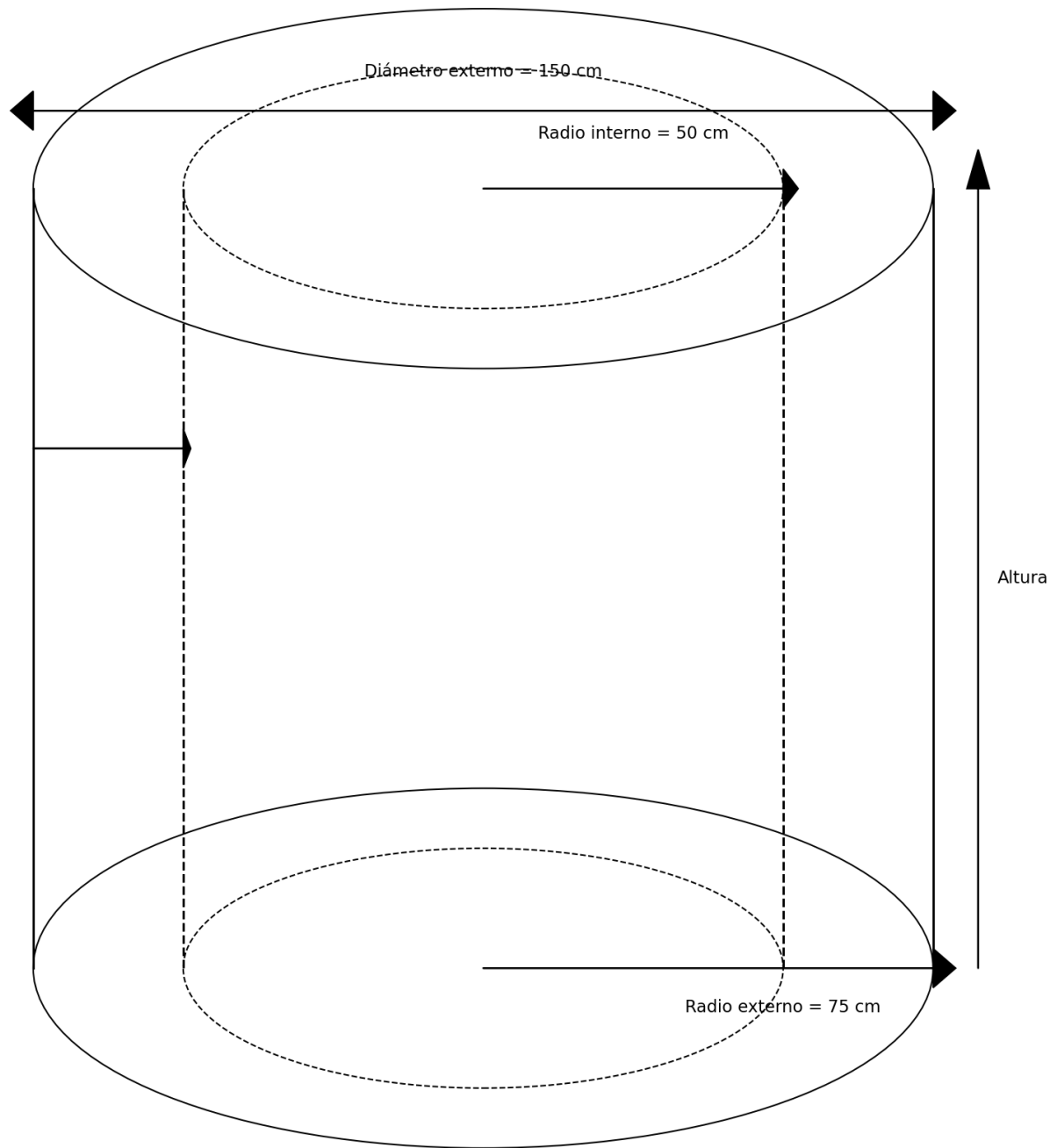


Figure 1: Cilindro hueco en 2D

Answerlist

- Radio externo.
- Diámetro externo.
- Altura del cilindro.
- Perímetro del cilindro.

Solution

La respuesta correcta es: **Altura del cilindro.**

Para calcular el volumen de aceite necesario para llenar el tanque cilíndrico interno, necesitamos calcular el volumen de un cilindro, cuya fórmula es:

$$V = \pi \cdot r^2 \cdot h$$

Donde: - r es el radio (en este caso, el radio interno = 50 cm) - h es la altura del cilindro

En el problema se nos proporciona: - Diámetro externo = 150 cm - Radio interno = 50 cm

Sin embargo, no se nos proporciona la altura del cilindro, que es esencial para calcular el volumen. Por lo tanto, a Carlos le falta conocer la altura del cilindro para poder calcular la cantidad de aceite necesaria.

Si tuviéramos la altura, el volumen se calcularía así: $V = \pi \cdot (50)^2 \cdot h = 7853,98 \cdot h \text{ cm}^3$

Answerlist

- Falso
- Falso
- Verdadero
- Falso

Meta-information

exname: volumen_cilindro_hueco extype: schoice exsolution: 0010 exshuffle: TRUE exsection: Geometría|Volumen|Cilindro